

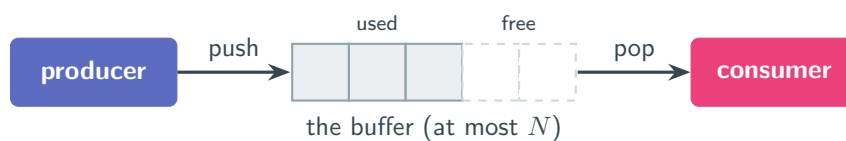
Task 8: The video decoder

Task 8 is about a famous problem of concurrency. Read this introduction first.

The producer-consumer problem

The **producer-consumer problem**, also called the **bounded buffer problem**, is one of the classic problems of concurrent programming. Edsger Dijkstra described it in 1965.

Some threads **produce** things (the **producers**), other threads **use** them (the **consumers**). They work at different speeds, so between them there is a **buffer**: a queue that holds at most N things.



There are two kinds of waiting:

- **The buffer is full:** the producer waits until the consumer takes something.
- **The buffer is empty:** the consumer waits until the producer adds something.

With mutexes only, the program is correct: the mutex protects the queue, so nothing is lost or broken. But a mutex cannot put a thread to sleep until something changes. A thread that waits for a full or an empty buffer has to unlock, lock and look again in a loop (busy waiting), and it uses the processor for nothing. **Condition variables do not fix a bug here, they make the waiting cheap:** the result is the same, but the waiting thread sleeps. You need three things:

a mutex	protects the queue, because both threads change it
not_full	a condition variable: the producer waits on it when the buffer is full; the consumer wakes it up after pop()
not_empty	a condition variable: the consumer waits on it when the buffer is empty; the producer wakes it up after push()

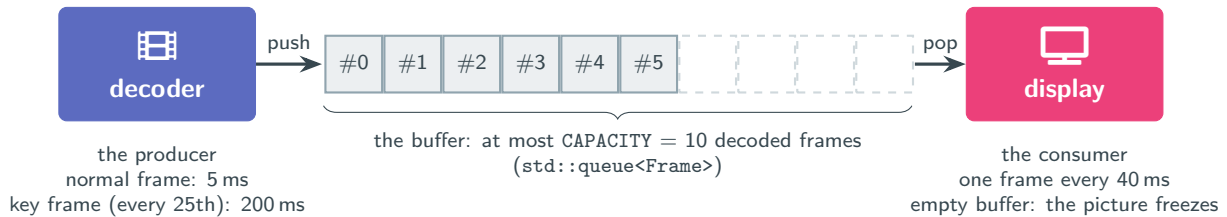
Why a buffer at all? It smooths out the differences in speed. When the producer is slow for a moment, the consumer takes from the stock in the buffer. When the consumer is slow for a moment, the producer puts things into the buffer and does not have to wait.

This pattern is everywhere: a video player, a printer queue, a web server that handles requests, and the pipe in a terminal (`cat file | grep word`: `cat` produces lines, `grep` consumes them).

Producer-consumer: two kinds of waiting (full, empty) around one buffer, protected by one mutex.

The task

You work for a company that writes a driver for video decoding. One thread decodes the frames of a video (the producer), and another one shows them on the screen at a fixed speed: one frame every 40 ms, 25 frames per second (the consumer). Decoding takes an uneven time: a normal frame is fast, a key frame (an I-frame) is much slower. Between the threads there is a buffer of decoded frames. When the buffer is empty, the picture freezes. The driver works correctly, but customers complain that their laptops get hot and the battery runs down fast. Your team must fix it.



Ready code (in `video.h`, we do not look inside)

Functions

<code>decode_frame(i)</code>	decode frame <code>i</code> : a normal frame takes 5 ms, every 25th frame is a key frame and takes 200 ms
<code>show(f)</code>	show the frame for 40 ms; if it comes more than 10 ms late, print that the picture froze
<code>report()</code>	print how many times the picture froze

Constants

<code>FRAMES</code>	100 frames: the video lasts 4 s
---------------------	---------------------------------

The template is in [Task8](#). This is `video.cpp`:

```

const int CAPACITY = 10;    // the buffer holds at most 10 frames

std::queue<Frame> buffer;
std::mutex m;

void decoder() {            // the producer
    for (int i = 0; i < FRAMES; i++) {
        Frame f = decode_frame(i);
        m.lock();
        while ((int)buffer.size() == CAPACITY) {    // full? look again
            m.unlock();
            m.lock();
        }
        buffer.push(f);
        m.unlock();
    }
    printf("decoder: all frames decoded\n");
}

void display() {            // the consumer
    while (true) {
        m.lock();
        while (buffer.empty()) {                    // empty? look again
            m.unlock();
            m.lock();
        }
        Frame f = buffer.front();
        buffer.pop();
        m.unlock();
        show(f);
    }
}

int main() {
    std::thread d(decoder), s(display);
    d.join();
    s.join();
    report();
}

```

In the task folder, make run compiles the program and runs it.

Your tasks

- 0 Who is the producer here, who is the consumer, and what is the buffer? When does each of them wait?
- 1 Run it with `time ./build/video`. How much processor time (user) do the threads use? (The program does not end: stop it with Ctrl+C about 2s after decoder: all frames decoded. Question 4 fixes this.)
- 2 Replace busy waiting with two condition variables, `not_empty` and `not_full`. Who waits on which one, and who wakes up whom? Run it with `time` again.
- 3 A colleague put `std::this_thread::sleep_for(std::chrono::milliseconds(10))` into the waiting loops instead. The laptop stays cool. Is this as good a solution as a condition variable?

- 4 The video is over, but the program does not end. Why? Fix it, so that the display shows all frames that are still in the buffer and then ends. (Hint: the decoder must tell the display that nothing more comes.)
- 5 Set `CAPACITY` to 1, then 10, then 100. How many times does the picture freeze? Why does the decoder work ahead? What does a larger buffer cost?